PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS — PUC MINAS

DISCIPLINA: QUALIDADE E TESTES DE SOFTWARE

Detecção de Bad Smells e Refatoração Segura

Análise estática de código com ESLint e SonarJS, identificação de débitos técnicos e refatoração arquitetural orientada a padrões de projeto sob a proteção de testes automatizados.

ESLint + SonarJS | Complexidade Cognitiva | Padrão Strategy & Polimorfismo | Rede de Segurança Jest

Aluno:	Luis Henrique Gonçalves Barbosa
Matrícula:	817204
Instituição / Curso:	PUC Minas — Engenharia de Software / Ciência da Computação
Repositório Fork:	https://github.com/BGLuis/bad-smells-js-refactoring
Repositório Base:	https://github.com/CleitonSilvaT/bad-smells-js-refactoring

BELO HORIZONTE — 2026

1 Análise Detalhada dos Bad Smells no Código de Produção

A inspeção estática manual do arquivo `src/ReportGenerator.js` revelou sintomas clássicos de degradação de design estrutural catalogados por Martin Fowler. Embora o código atendesse aos testes iniciais, os **Bad Smells** comprometiam severamente a legibilidade, modularidade e testabilidade da regra de negócio:

1.1. Long Method (Método Longo) e Violação de SRP

O que é e onde ocorre: O método `generateReport` concentra em uma única função de 69 linhas todas as responsabilidades do domínio: chaveamento de formato (CSV e HTML), cabeçalhos, filtragem por perfil (ADMIN vs USER), mutação direta de atributos (`item.priority`), cálculo acumulado de totais e renderização de rodapé.

Por que é problemático: Viola o Princípio da Responsabilidade Única (SRP). Concentrar geração de leiaute, autorização de acesso e cálculo financeiro em um só ponto impede a reutilização e exige alterar a classe para qualquer mudança de leiaute.

Impacto na manutenção e testes: Eleva a complexidade ciclomática e gera uma matriz de testes combinatória (formatos × papéis). Dificulta testar o cálculo financeiro ou as regras de perfil isoladamente sem renderizar strings de saída completas.

1.2. Alta Complexidade Cognitiva e Aninhamento Profundo (Deeply Nested Logic)

O que é e onde ocorre: No laço de iteração dos itens, há aninhamento de 4 níveis de desvios condicionais: `for -> if (user.role === 'ADMIN') -> if (item.value > 1000) -> if (reportType === 'CSV')` .

Por que é problemático: A métrica de complexidade cognitiva do SonarSource mede a sobrecarga mental para rastrear o fluxo lógico. A função original obteve pontuação **27**, ultrapassando amplamente o teto seguro de **15** estipulado no trabalho.

Impacto na manutenção e testes: Multiplica a chance de regressões ao alterar regras de negócio. Caminhos de execução alternativos ficam obscuros, gerando pontos cegos na suíte de testes (dificuldade de cobrir 100% dos branches).

1.3. Código Duplicado (Duplicated Code) e Falta de Abstração

O que é e onde ocorre: A concatenação de colunas CSV (`${item.id},${item.name},${item.value},${user.name}\n`) e a soma cumulativa `total += item.value;` aparecem replicadas linha a linha nos blocos de `ADMIN` e `USER` . Além disso, o chaveamento `reportType` se repete em 3 pontos distintos (cabeçalho, corpo e rodapé).

Por que é problemático: Viola o princípio *Don't Repeat Yourself (DRY)* e indica ausência de polimorfismo para os formatos de saída.

Impacto na manutenção e testes: Se a ordem ou formato de uma coluna CSV mudar, a alteração precisa ser replicada manualmente em múltiplos locais, sob risco iminente de divergência entre relatórios de ADMIN e USER em produção.

Bad Smell Identificado	Regra SonarJS / ESLint Associada	Severidade	Impacto Principal no Software
Alta Complexidade Cognitiva	`sonarjs/cognitive-complexity`	Erro	Sobrecarga de 27 pontos vs limite 15; aninhamento de 4 níveis de if/else
Condicionais Aninhadas Redundantes	`sonarjs/no-collapsible-if`	Erro	Branches colapsáveis desnecessários poluindo a árvore de decisão
Violação de SRP & Método Longo	`max-lines-per-function` / `SonarJS`	Aviso / Débito	Método acumulando 5 papéis de domínio distintos sem coesão
Código Duplicado (DRY)	`sonarjs/no-identical-functions`	Erro / Alerta	Formatação CSV e soma repetidas em ramos distintos de autorização

2 Processo de Refatoração Segura e Aplicação de Padrões

Para erradicar os problemas diagnosticados sem quebrar nenhum contrato de negócio, criou-se o arquivo `src/ReportGenerator.refactored.js`. O processo foi conduzido sob o catálogo canônico de Martin Fowler, substituindo estruturas imperativas aninhadas por padrões modulares orientados a objetos.

2.1. Estudo de Caso: Eliminação do Acoplamento de Formatos (Padrão Strategy / Polimorfismo)

× ANTES: SRC/REPORTGENERATOR.JS (COMPLEXIDADE 27)

```
// Laço com 4 níveis de if e mutação direta
for (const item of items) {
  if (user.role === 'ADMIN') {
    if (item.value > 1000) item.priority = true;
    if (reportType === 'CSV') {
      report +=
`${item.id},${item.name},${item.value},${user.name}\n`;
      total += item.value;
    } else if (reportType === 'HTML') {
      const style = item.priority
        ? ' style="font-weight:bold;"' : '';
      report += `<tr${style}><td>${item.id}</td>...`;
      total += item.value;
    }
  } else if (user.role === 'USER') {
    if (item.value <= 500) {
      if (reportType === 'CSV') {
        report +=
`${item.id},${item.name},${item.value},${user.name}\n`;
        total += item.value;
      } else if (reportType === 'HTML') {
        report += `<tr><td>${item.id}</td>...`;
        total += item.value;
      }
    }
  }
}
```

✓ DEPOIS: SRC/REPORTGENERATOR.REFACTORED.JS (COMPLEXIDADE ≤ 2)

```
// Coordenação desacoplada com Strategy
generateReport(reportType, user, items) {
  const formatter = this._resolveFormatter(reportType);
  const visibleItems = this._filterAndProcessItems(user,
items);
  const total = this._calculateTotal(visibleItems);
  return this._renderReport(formatter, user, visibleItems,
total);
}

// Decompose Conditional e Imutabilidade Funcional
_filterAndProcessItems(user, items) {
  if (user.role === ROLES.ADMIN) {
    return items.map(item => ({
      ...item,
      priority: item.value > THRESHOLDS.ADMIN_PRIORITY
    }));
  }
  if (user.role === ROLES.USER) {
    return items.filter(item => item.value <=
THRESHOLDS.USER_MAX_VALUE);
  }
  return [];
}
```

2.2. Decisões Técnicas de Refatoração e Justificativas

- **Replace Conditional with Strategy / Polymorphism:** As variações de cabeçalho, linha e rodapé dos formatos CSV e HTML foram extraídas para classes polimórficas (`CsvReportFormatter` e `HtmlReportFormatter`). Com isso, eliminou-se o chaveamento condicional espalhado e a aplicação abriu-se para extensões futuras sem alteração no núcleo (Open/Closed Principle).
- **Extract Method (Extração de Método):** O processamento foi segmentado em responsabilidades atômicas com escopos únicos: `_resolveFormatter` , `_filterAndProcessItems` , `_calculateTotal` e `_renderReport` . Cada método possui complexidade cognitiva ≤ 2.
- **Decompose Conditional e Imutabilidade:** As condições de filtragem e atribuição de prioridade foram isoladas com métodos funcionais nativos (`map` , `filter` , `reduce`), extinguindo a mutação in-place perigosa dos objetos de dados originais.
- **Replace Magic Numbers with Named Constants:** Literais mágicos soltos no código (`1000` e `500`) foram formalizados nos dicionários imutáveis `THRESHOLDS.ADMIN_PRIORITY` e `THRESHOLDS.USER_MAX_VALUE` (Object.freeze).
- **Preservação Estrita do Comportamento Observável:** A suíte de testes original e a suíte espelho foram executadas continuamente a cada transformação, garantindo paridade total nas saídas de dados para perfis ADMIN, USER comum e casos de borda com array vazio.

3 Diagnóstico do ESLint, Rede de Segurança e Conclusão

3.1. Execução Inicial do ESLint + SonarJS no Código Original (Detecção Automatizada)

Com a configuração estrita das regras no arquivo `.eslintrc.json` contendo o plugin `eslint-plugin-sonarjs` e o teto de complexidade cognitiva fixado em **15**, o linter detectou objetivamente as seguintes falhas estruturais em `src/ReportGenerator.js`:

```
/home/luis/Documents/puc/testes/bad-smell/src/ReportGenerator.js
  11:3   error  Refactor this function to reduce its Cognitive Complexity from 27 to the 15 allowed  sonarjs/cognitive-complexity
  43:14  error  Merge this if statement with the nested one                                          sonarjs/no-collapsible-if

✖ 2 problems (2 errors, 0 warnings)
```

Como o SonarJS automatizou a detecção: Enquanto linters convencionais avaliam formatação léxica e erros sintáticos, o SonarJS constrói a Árvore Sintática Abstrata (AST) do fluxo de controle. A complexidade cognitiva soma pontos incrementais ponderados por nível de aninhamento. Cada condicional aninhada no laço gerou penalidades cumulativas, revelando que a função acumulava **27 pontos de complexidade**, tornando imediata a localização do colapso de legibilidade.

3.2. Validação da Suíte Refatorada

Após a aplicação das técnicas de refatoração, executou-se a validação cruzada do código refatorado e das suítes de testes:

> **✓ Conformidade Plena: Zero Violações de Linter e 100% de Testes Verdes**
>
> Execução de `npx eslint src/ReportGenerator.refactored.js`: **0 erros, 0 avisos (código 100% limpo)**.
> Execução de `npm test`: **4 test suites executadas com 100% de aprovação (20 testes passando, 0 quebras)**.

Métrica de Qualidade	Código Original (ReportGenerator.js)	Código Refatorado (ReportGenerator.refactored.js)
Complexidade Cognitiva	27 (Violação de regra: teto permitido 15)	≤ 2 por método (100% dentro dos limites)
Erros de Linter (ESLint/SonarJS)	2 erros bloqueantes detectados	0 erros, 0 avisos
Aderência ao Princípio SRP	Baixa (1 classe acumulando 5 responsabilidades)	Alta (Formatadores isolados + Gerador coordenador)
Preservação de Testes (Rede de Segurança)	Original com inconsistência em espaçamento	100% de sucesso em todos os cenários (Admin, User, Vazio)

3.3. Conclusão e Reflexão

O desenvolvimento deste trabalho demonstrou na prática o valor indispensável de utilizar uma **suíte de testes automatizados como rede de segurança**. A segurança em transformar um código densamente acoplado em um design limpo baseado em Strategy e Polimorfismo adveio da garantia de que nenhuma regressão seria tolerada pelo Jest a cada extração de método.

Aliar a análise estática avançada (ESLint + SonarJS) aos testes de regressão consolida um ciclo moderno de engenharia de software: ferramentas inteligentes detectam a degradação de design prematuramente, métricas matemáticas quantificam a sobrecarga cognitiva e testes automatizados viabilizam a refatoração contínua sem interrupção de valor para o negócio.